

Image Upload Functionality in a Node.js API Using Multer

1. Introduction

This tutorial explains how to integrate **image upload functionality** into a Node.js CRUD API. It includes schema configuration, Multer setup, upload handling, update logic, deletion logic, and error handling.

2. Project Structure and Existing Setup

2.1 Installed Packages

- express
- mongoose
- nodemon
- multer (installed during this tutorial)

2.2 Folder Structure

```
/crud-api-project
  /uploads
  /routes/student.routes.js
  /config/db.js
  /models/student.model.js
  index.js
  package.json
```

2.3 Student Model

```
const StudentSchema = new mongoose.Schema({
  firstName: String,
  lastName: String,
  email: String,
  phone: String,
  gender: String,
  profilePic: { type: String } // Image filename will be saved here
});
```

3. Installing and Configuring Multer

3.1 Install Multer

```
npm install multer
```

3.2 Import Required Modules

```
const multer = require("multer");  
const path = require("path");
```

4. Multer Configuration

4.1 Storage Engine

Defines upload directory and file renaming logic.

```
const storage = multer.diskStorage({  
  destination: (req, file, cb) => {  
    cb(null, "uploads");  
  },  
  filename: (req, file, cb) => {  
    const newName = Date.now() + path.extname(file.originalname);  
    cb(null, newName);  
  }  
});
```

4.2 File Filter (Allow Only Images)

```
const fileFilter = (req, file, cb) => {  
  if (file.mimetype.startsWith("image")) {  
    cb(null, true);  
  } else {  
    cb(new Error("Only images are allowed"), false);  
  }  
};
```

4.3 Size Limit

```
const upload = multer({  
  storage,  
  fileFilter,
```

```
limits: { fileSize: 3 * 1024 * 1024 } // 3 MB
});
```

5. Adding Image Upload in Create API (POST /students)

```
router.post(
  "/students",
  upload.single("profilePic"), // middleware
  async (req, res) => {
    const student = new Student(req.body);

    if (req.file) {
      student.profilePic = req.file.filename;
    }

    const result = await student.save();
    res.json(result);
  }
);
```

Testing Notes

Use Postman → Body → Form-data Fields:

- firstName (text)
- lastName (text)
- email (text)
- phone (text)
- gender (text)
- profilePic (file)

6. Deleting Image When Student Record Is Deleted

6.1 Import File System

```
const fs = require("fs");
```

6.2 Delete Route Logic

```
router.delete("/students/:id", async (req, res) => {
  const student = await Student.findByIdAndDelete(req.params.id);

  if (!student) {
    return res.json({ message: "Student not found" });
  }

  if (student.profilePic) {
    const filePath = path.join("uploads", student.profilePic);
    fs.unlink(filePath, err => {
      if (err) console.log("Failed to delete image", err.message);
    });
  }

  res.json({ message: "Student deleted" });
});
```

7. Updating a Student with New Image (PUT /students/:id)

7.1 Add Middleware for Image Upload

```
router.put(
  "/students/:id",
  upload.single("profilePic"),
  async (req, res) => {
    const existingStudent = await Student.findById(req.params.id);

    if (!existingStudent) {
      // Delete mistakenly uploaded file
      if (req.file) {
        const pathToDelete = path.join("uploads", req.file.filename);
        fs.unlink(pathToDelete, () => {});
      }
      return res.json({ message: "Student not found" });
    }

    if (req.file) {
      // delete old image
      if (existingStudent.profilePic) {
        const oldPath = path.join("uploads", existingStudent.profilePic);
        fs.unlink(oldPath, () => {});
      }
      req.body.profilePic = req.file.filename;
    }

    const updated = await Student.findByIdAndUpdate(
```

```
    req.params.id,  
    req.body,  
    { new: true }  
  );  
  
  res.json(updated);  
}  
);
```

8. Error Handling Middleware

Placed in **index.js** below all routes.

```
app.use((err, req, res, next) => {  
  if (err instanceof multer.MulterError) {  
    return res.status(400).json({  
      errorType: "Multer Error",  
      message: err.message  
    });  
  }  
  
  return res.status(500).json({  
    errorType: "Server Error",  
    message: err.message  
  });  
});
```

9. Displaying Uploaded Images in Twig Templates

If UI displays student details, Twig syntax would be:

```

```

10. Final Implementation Capabilities

This API now supports:

1. **Uploading images** when creating a student
2. **Renaming and storing** image file names
3. **Deleting images** when the student record is deleted
4. **Updating images**, including removal of the old image

5. **Preventing orphaned images** when invalid operations occur
6. **Centralized error handling** for Multer and server errors

The API is now fully equipped for real-world front-end consumption.
